SZKOLENIE:Modularny monolit

COMARCH SA CENTRUM SZKOLENIOWO-KONFERENCYJNE

Leszek Lewandowski

Senior Fullstack developer - DNV

Start szkolenia: ***9:00***

COMARCH

Wprowadzenie

- Trener
- Współpraca
- Przerwy

Kontakt

LinkedIn

https://www.linkedin.com/in/leszek-lewandowski-nadalasc

WWW

http://lasoty.pl/

E-mail

Leszek.Lewandowski@gmail.com

COMARCH

Rozdział 1

Podstawy architektury modularnej

COMARCH

Monolit — co to właściwie znaczy?

Pojęcie

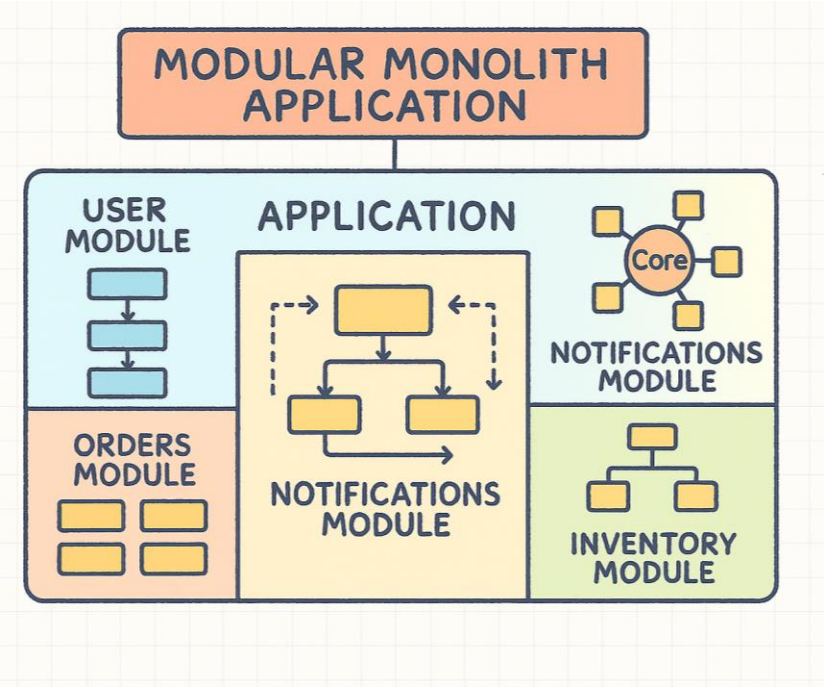
Monolit to aplikacja wdrażana jako jedna całość. Może być dobrze uporządkowana albo chaotyczna — sam fakt wspólnego wdrożenia nie przesądza jeszcze o jakości architektury.

Typowe cechy

- jeden proces / jeden artefakt wdrożeniowy
- zwykle jedna baza albo wspólne zarządzanie danymi
- silna wygoda startowa
- ryzyko narastania sprzężeń

Podstawy architektury modularnej

Najpierw ustalamy, czym jest moduł i po co w ogóle chcemy budować granice wewnątrz jednej aplikacji.



Czym jest moduł?

Moduł to przede wszystkim granica odpowiedzialności, a dopiero później fragment kodu.

Moduł powinien mieć

- wyraźny obszar biznesowy
- własne reguły i własne use case'y
- jawny sposób komunikacji z innymi
- kontrolę nad swoimi danymi
- możliwość rozwoju bez ciągłego dotykania całego systemu

Dobra intuicja

Moduł to nie jest folder. To umowa architektoniczna: co jest w środku, co pokazujemy na zewnątrz i czego innym robić nie wolno.

Najczęstsza pomyłka

System bywa podzielony na projekty, ale wszystkie projekty nawzajem zaglądają sobie do środka. Wtedy modularność jest tylko dekoracją.

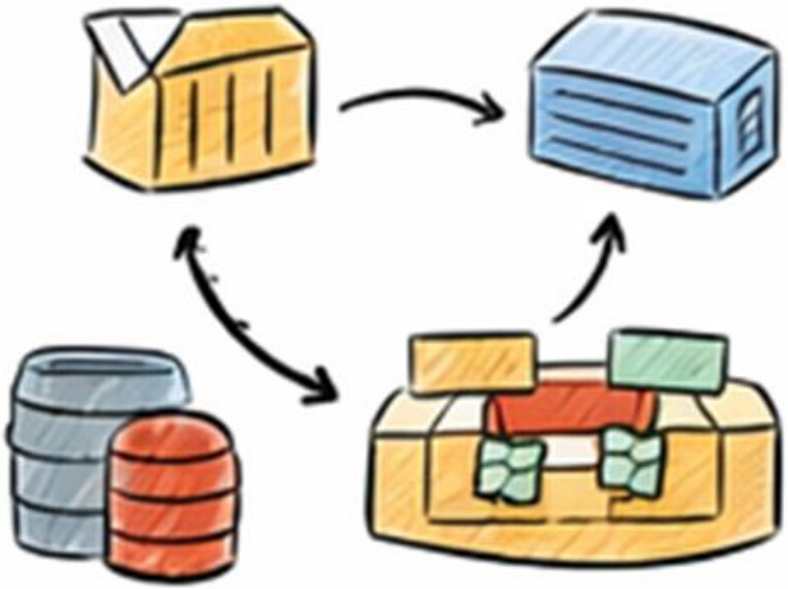
Na co patrzymy przy wyznaczaniu modułu

- język biznesowy i odpowiedzialność
- częstość zmian
- granice danych
- potrzebę niezależnego skalowania procesu
- możliwość późniejszego wydzielenia

Antywzorce modularności — szczególnie wokół danych

Najczęstsze błędy:

- moduł A robi joiny po tabelach modułu B
- wspólne encje rozlane po całym systemie
- jeden DbContext dla całego świata
- raportowanie mieszane z transakcyjnym modelem domenowym
- „tymczasowe” skróty, które zostają na lata



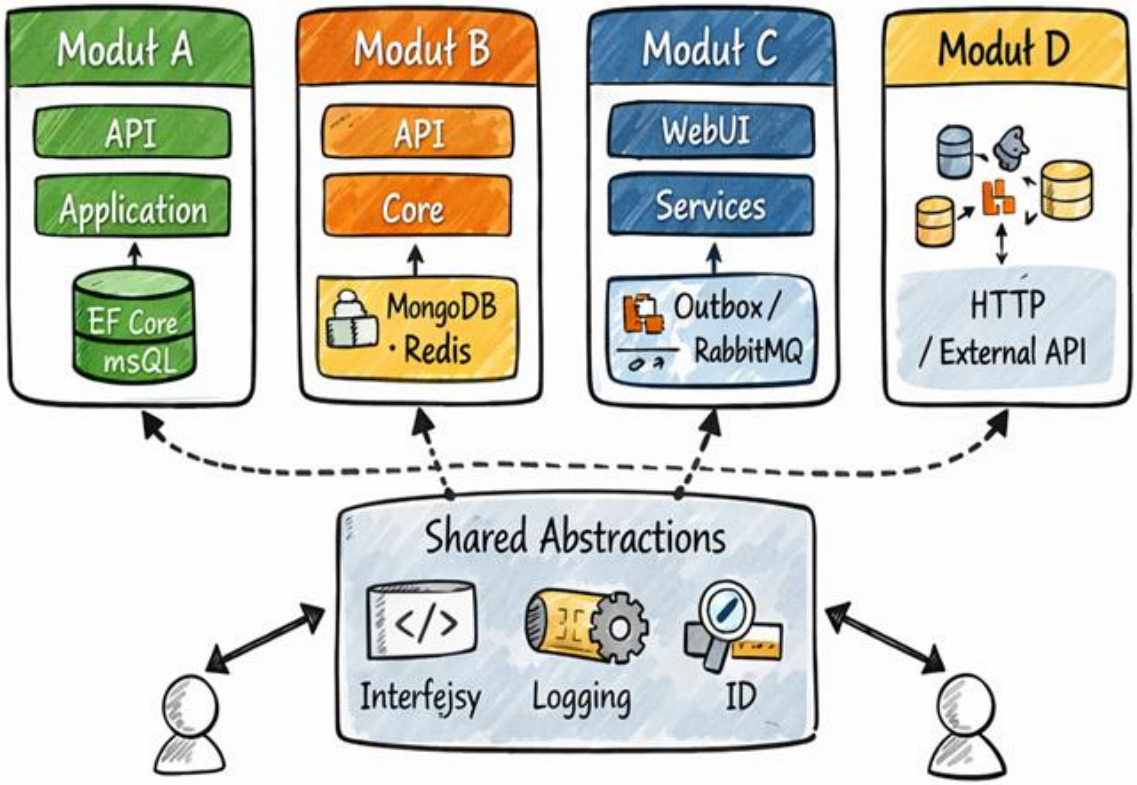
Rozdział 2

Projektowanie modularnego monolitu

COMARCH

Przykładowy układ modułów — architektura może się różnić

Wspólny host aplikacji, ale moduły nie muszą mieć identycznego wnętrza.



COMARCH

Żelazne reguły modularnego monolitu

- Każdy moduł ma własną odpowiedzialność biznesową
- Komunikacja tylko przez publiczne API albo zdarzenia
- Brak bezpośrednich odwołań do wewnętrznych klas innych modułów
- Dane modułu nie są czytane i zapisywane na skróty
- Kontrakty są małe, jawne i stabilne
- Współdzielimy minimum, nie całą implementację

Różne bazy danych i różne modele przechowywania

W jednym modularnym monolicie jeden moduł może korzystać z relacyjnej bazy transakcyjnej, inny z dokumentowej, a jeszcze inny z wyszukiwarki lub osobnego modelu read-side. Ważna jest granica odpowiedzialności, nie jednolitość narzędzia.

Publiczne API modułu

Każdy moduł powinien jawnie określać, jak inni mogą z niego korzystać.

Co może należeć do publicznego API

- komendy i query
- interfejs fasady aplikacyjnej
- kontrakty odpowiedzi
- zdarzenia publikowane przez moduł
- minimalny zestaw wyjątków i kodów błędów

Przykład kodu — kontrakt publicznego API modułu

Prosty, jawny kontrakt jest lepszy niż wpuszczanie innych modułów do środka implementacji.

```
1  namespace Orders.Contracts;
2
3  public interface IOrdersModule
4  {
5      Task<OrderDto?> GetByIdAsync(Guid orderId, CancellationToken ct);
6      Task<Guid> PlaceOrderAsync(PlaceOrderCommand command, CancellationToken ct);
7  }
8
9  public sealed record PlaceOrderCommand(
10     Guid CustomerId,
11     IReadOnlyList<OrderLineDto> Lines
12 );
```

Rozdział 3

Współdzielone abstrakcje i infrastruktura

COMARCH

Co można współdzielić, a czego lepiej nie?

Najczęściej warto współdzielić:

- kontrakty komunikacyjne
- typy infrastrukturalne: clock, user context, correlation
- mechanizmy rejestracji modułów
- wspólne pipeline'y techniczne
- podstawowe biblioteki cross-cutting

Lepiej nie współdzielić:

- encji i agregatów
- wspólnego DbContextu
- modeli ORM i mapowań
- „uniwersalnych" serwisów biznesowych
- klas pomocniczych, które tak naprawdę niosą logikę domenową

Rozdział 4

Wewnętrzna komunikacja między modułami

COMARCH

Modele komunikacji między modułami

Nie ma jednego dobrego stylu — dobieramy podejście do typu współpracy i oczekiwanej niezależności.

Wywołanie przez API modułu

Najprostsze podejście, dobre gdy potrzebujemy odpowiedzi od razu.
Plusy: prostota, czytelny przepływ.
Minusy: większe sprzężenie czasowe.

Komenda przez mediator / bus

Użyteczne, gdy chcemy ujednolicić obsługę komend, pipeline i cross-cutting concerns.
Plusy: spójny model.
Minusy: łatwo ukryć zbyt dużo magii.

Zdarzenie domenowe lub integracyjne

Dobre, gdy ktoś ma zareagować na fakt biznesowy bez twardego sprzężenia.
Plusy: rozszerzalność.
Minusy: większa złożoność i eventual consistency.

COMARCH

Service Bus, event broker, Kafka — o czym mówimy?

- **Service Bus**
  - Model bardziej komendowy i integracyjny. Dobrze pasuje do kolejek roboczych, workflow oraz asynchronicznego wywoływania operacji. Typowe przykłady: Azure Service Bus, RabbitMQ z warstwą aplikacyjną.
- **Event broker**
  - Model publikacja–subskrypcja. Interesuje nas fakt, że coś się wydarzyło, a nie bezpośredni odbiorca. Dobre do luźnego sprzężenia i rozbudowy reakcji.
- **Kafka / log zdarzeń**
  - Myślenie strumieniowe i trwały log zdarzeń. Szczególnie przydaje się przy dużej skali, analityce, replay oraz wielu konsumentach. Nie zawsze jest pierwszym wyborem do prostego modularnego monolitu.

Przykład kodu — publikacja zdarzenia z modułu

Zdarzenie powinno opisywać fakt biznesowy, a nie techniczny szczegół implementacji.

```
1  public sealed record OrderPlaced(
2      Guid OrderId,
3      Guid CustomerId,
4      DateTime OccurredAtUtc
5  );
6
7  public async Task<Guid> PlaceOrderAsync(PlaceOrderCommand command, CancellationToken ct)
8  {
9      var order = Order.Create(command.CustomerId, command.Lines);
10     _dbContext.Orders.Add(order);
11
12     _outbox.Add(new OrderPlaced(order.Id, order.CustomerId, _clock.UtcNow));
13     await _dbContext.SaveChangesAsync(ct);
14     return order.Id;
15 }
```

Rozdział 5

Architektura sterowana zdarzeniami i asynchroniczność

COMARCH

EDA w praktyce

Dobrze rozumiane zdarzenie

- nazywa fakt biznesowy
- jest niemodyfikowalne
- jest publikowane po zmianie stanu
- może mieć wielu odbiorców
- nie powinno zdradzać zbyt wielu szczegółów wewnętrznych

Złe użycie

- zdarzenie jako ukryta komenda
- techniczne nazwy bez znaczenia biznesowego
- zbyt grube payloady
- brak wersjonowania kontraktu

Praktyczny kompromis

W modularnym monolicie część zdarzeń może być jeszcze in-process, ale warto projektować je tak, jakby kiedyś mogły wyjść poza proces.

Asynchroniczne przetwarzanie zdarzeń i jego zastosowania

Asynchroniczność najczęściej wprowadzamy po coś konkretnego, a nie dlatego, że brzmi nowocześnie.

Po co to robimy?

- odciążenie ścieżki użytkownika
- rozbudowa procesu o kolejne reakcje
- integracja z systemami zewnętrznymi
- oddzielenie odpowiedzialności między modułami

Co zyskujemy?

- mniejsze sprzężenie czasowe
- większą elastyczność rozwoju
- możliwość stopniowego przejścia do komunikacji zewnętrznej

Co płacimy?

- eventual consistency
- retry i duplikaty
- trudniejsze śledzenie przepływu
- większą złożoność testów

Kluczowe wzorce transakcyjne dla niezawodności systemu

Nie da się bezpiecznie połączyć zapisu danych i publikacji wiadomości bez świadomego wzorca.

zmiana stanu własne dane

zapis wiadomości w tej samej transakcji

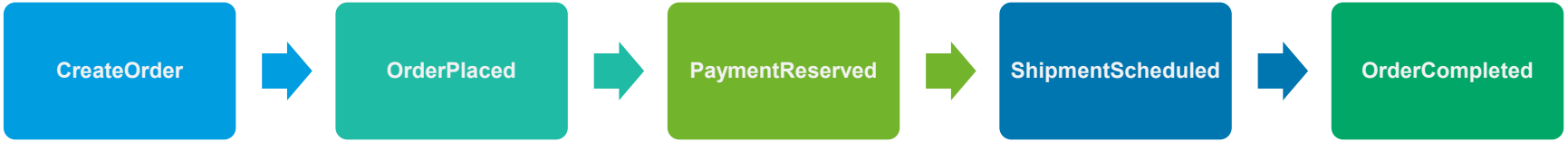
publikacja poza transakcją z retry

odporność na duplikaty i ponowienia

Brak globalnej transakcji nadrabiamy wzorcami niezawodności.

Korelacja wiadomości i zarządzanie przepływem

Gdy proces rozciąga się na kilka kroków, musimy umieć go śledzić i rozumieć.



Correlation ID spina wszystkie kroki jednego procesu. Causation ID pozwala zrozumieć, która wiadomość wywołała kolejną.

Saga — obsługa złożonych procesów biznesowych

Saga pomaga wtedy, gdy proces ma wiele kroków, stan pośredni i ewentualne kompensacje.

Kiedy Saga ma sens

- proces obejmuje kilka modułów
- nie mamy jednej wspólnej transakcji
- musimy śledzić stan pośredni
- potrzebujemy reakcji kompensacyjnych

Dwa style

Choreografia: moduły reagują na zdarzenia.

Orkiestracja: osobny koordynator pilnuje kolejnych kroków.

W praktyce wybór zależy od złożoności procesu i potrzeby centralnej kontroli.

Na co uważać

- zbyt dużo ukrytej logiki w handlerach
- brak jawnego modelu stanu
- brak timeoutów i kompensacji
- trudna obserwowalność

Przykład kodu — szkic orkiestratora Sagi

Nie chodzi o pełny framework, tylko o pokazanie, że proces można modelować jawnie.

```
1  public sealed class OrderSaga
2  {
3      public async Task HandleAsync(OrderPlaced message, CancellationToken ct)
4      {
5          await _bus.SendAsync(new ReservePayment(message.OrderId), ct);
6          await _stateStore.SetStatusAsync(message.OrderId, "PaymentPending", ct);
7      }
8
9      public async Task HandleAsync(PaymentFailed message, CancellationToken ct)
10     {
11         await _bus.SendAsync(new CancelOrder(message.OrderId), ct);
12         await _stateStore.SetStatusAsync(message.OrderId, "Compensated", ct);
13     }
14 }
```

Strategia przejścia z modularnego monolitu do mikroserwisów

Najpierw separacja logiczna, dopiero potem decyzja o separacji runtime'owej.

Nie wydzielamy usług dlatego, że „tak się teraz robi". Wydzielamy je wtedy, gdy moduły mają dojrzałe granice i realną potrzebę operacyjnej niezależności.

Podsumowanie

- Najważniejsze wnioski
  - modularny monolit to nie etap przejściowy z definicji, ale bardzo dobry etap dojrzewania systemu
  - granice modułów muszą być prawdziwe
  - dane są najczęstszym miejscem łamania architektury
- Co daje największy zwrot
  - publiczne API modułów
  - zdarzenia projektowane świadomie
  - outbox, idempotencja i korelacja
  - ostrożne współdzielenie
- W praktyce
  - Zaczynamy od czytelnych granic i rozsądnej prostoty. Dopiero później dokładamy bardziej zaawansowane wzorce tam, gdzie faktycznie są potrzebne.

Kontakt

LinkedIn

https://www.linkedin.com/in/leszek-lewandowski-nadalasc

WWW

http://lasoty.pl/

E-mail

Leszek.Lewandowski@gmail.com

COMARCH

DZIĘKUJEMY

Centrum Szkoleniowe
ul. Prof. M.Życzkowskiego 23
31-864 Kraków
Tel. +48 (12) 687 78 11
E-Mail: mail.szkolenia@comarch.pl
www.szkolenia.comarch.pl

COMARCH